



PYTHON — Exercices guidés

Exercices 12 à 25 — Fonctions, booléens, boucles avancées

Version avec méthode pas à pas — pour ceux qui découvrent Python



Comment utiliser cette feuille

Chaque exercice est accompagné d'un encadré  Méthode qui te guide étape par étape, sans te donner directement la solution : lis chaque étape, réfléchis, essaie, puis passe à la suivante.

Le cadre gris est là pour que tu écrives ton code ou ta réponse. N'hésite pas à le tester ensuite sur ordinateur pour vérifier.

Si une étape te semble difficile, relis d'abord ce que tu as écrit à l'étape précédente : la difficulté vient souvent d'un petit oubli plus tôt.

Exercice 12

a. Quelle est la valeur de a à la fin du programme suivant ? b. Qu'affiche ce programme ?

```
a = 21

def double(x):
    print(x * 2)

a = double(a)
```



Méthode, pas à pas

Étape 1 — Regarde la fonction double(x) : contient-elle une instruction return ? Repère bien la différence entre print (qui affiche à l'écran) et return (qui renvoie une valeur utilisable par le programme).

Étape 2 — Quand double(a) s'exécute, quelque chose s'affiche à l'écran : qu'est-ce que c'est, et à quel moment (avant ou après l'affectation à la ligne a = double(a)) ?

Étape 3 — Une fonction sans return renvoie toujours une valeur spéciale, quelle que soit son contenu. Quelle est cette valeur en Python ?

Étape 4 — La ligne a = double(a) affecte donc à a le résultat de l'étape 3, et non le double de 21. Quelle est la valeur finale de a ?

Valeur finale de a, et ce qui s'affiche à l'écran :

.....

.....

Exercice 13

Vérifie ces deux identités (lois de De Morgan) en écrivant un programme qui calcule les deux membres pour toutes les valeurs possibles de a et b, et qui affiche si l'égalité est vraie à chaque fois :

```
not (a and b) == not a or not b
not (a or b) == not a and not b
```



Méthode, pas à pas

Étape 1 — a et b sont des booléens : combien de couples (a, b) différents existe-t-il au total ? Essaie de tous les lister sur un brouillon avant de coder.

Étape 2 — Pour tester automatiquement toutes ces combinaisons sans les écrire une par une, quelle structure

permet de faire varier a, puis b, à l'intérieur ? (Indice : deux boucles for, l'une imbriquée dans l'autre, qui parcourent chacune la liste [True, False].)

Étape 3 — À l'intérieur des boucles, calcule séparément le membre de gauche et le membre de droite de la première identité, dans deux variables, puis compare-les avec ==.

Étape 4 — Affiche a, b, et le résultat de la comparaison à chaque tour de boucle. Fais de même pour la seconde identité.

Exercice 14

Écris une fonction xor(a, b) qui retourne le OU EXCLUSIF de a et b (vrai si a et b sont différents, faux s'ils sont identiques).

Méthode, pas à pas

Étape 1 — Construis la table de vérité à la main : que doit renvoyer xor(True, True) ? xor(True, False) ? xor(False, True) ? xor(False, False) ?

Étape 2 — Regarde ta table : dans quels cas renvoie-t-elle True ? Est-ce que ça correspond à un opérateur de comparaison que tu connais déjà, appliqué directement à a et b ?

Étape 3 — Écris la fonction avec un seul return, en utilisant cet opérateur — pas besoin de if !

Étape 4 — Teste ta fonction sur les quatre cas de l'étape 1 pour vérifier qu'elle correspond bien à ta table de vérité.

Exercice 15

a. Écris un programme qui indique si une année est bissextile.

Méthode, pas à pas

Étape 1 — Rappel de la règle : une année est bissextile si elle est divisible par 4, SAUF si elle est divisible par 100 — sauf si elle est ELLE-MÊME divisible par 400 (auquel cas elle redevient bissextile).

Étape 2 — Quel opérateur Python permet de tester « divisible par » ? (Indice : le reste d'une division.)

Étape 3 — Traduis la règle de l'étape 1 en une combinaison de and, or, not — ou en plusieurs if/elif imbriqués, comme tu préfères.

Étape 4 — Teste ton programme sur quelques années connues : 2024 (bissextile), 2023 (non), 1900 (non, piège du /100), 2000 (bissextile, piège du /400).

b. Écris un programme pour calculer le nombre de jours du mois m de l'année a.

Méthode, pas à pas

Étape 1 — Combien de jours ont les mois 1, 3, 5, 7, 8, 10, 12 ? Et les mois 4, 6, 9, 11 ? Le mois 2 est un cas particulier — lequel ?

Étape 2 — Une liste Python peut stocker les 12 valeurs dans l'ordre : jours = [31, 28, 31, ...]. Attention : le mois 1

(janvier) est à l'indice 0 de la liste !

Étape 3 — Pour février (mois 2, indice 1), il faut corriger la valeur 28 en 29 si l'année est bissextile — réutilise ta réponse de la question a.

Exercice 16

Reprends l'exercice 15 sous forme de fonctions réutilisables, puis va plus loin.

a. Écris les fonctions `est_bissextile(annee)` et `nb_jours_mois(mois, annee)`.

Méthode, pas à pas

Étape 1 — Reprends ton code de l'exercice 15a et transforme-le en fonction : ajoute `def est_bissextile(annee):` au-dessus, indente le corps, et remplace le print final par un `return` de la valeur booléenne (`True/False`).

Étape 2 — Fais de même pour `nb_jours_mois(mois, annee)` à partir de ton code de l'exercice 15b : cette fonction doit renvoyer (`return`) un nombre, pas l'afficher.

b. Écris une fonction `nb_jours(jour, mois, annee)` qui renvoie le nombre de jours écoulés depuis le début de l'année.

Méthode, pas à pas

Étape 1 — Pour compter les jours écoulés, as-tu besoin du nombre de jours de TOUS les mois avant le mois en cours ? Une boucle `for` qui parcourt les mois de 1 à `mois-1`, en additionnant `nb_jours_mois(...)` à chaque tour, peut faire ce travail.

Étape 2 — N'oublie pas d'ajouter le nombre jour du mois en cours à la fin (le nombre de jours écoulés dans le mois où l'on se trouve).

c. Écris une fonction `jour_semaine(jour, mois, annee, jour0)` qui renvoie le jour de la semaine.

Méthode, pas à pas

Étape 1 — `jour0` te donne le jour de la semaine du 1er janvier de l'année (par exemple 0 pour lundi). Comment le nombre de jours écoulés (calculé en b) permet-il de savoir de combien de jours on s'est décalé depuis le 1er janvier ?

Étape 2 — Il y a 7 jours dans une semaine : quel opérateur permet de « revenir au début » une fois qu'on dépasse 7 ? (Indice : le même que pour tester la divisibilité, mais utilisé différemment ici.)

Exercice 17

a. Boucle for qui affiche les nombres pairs entre 2 et 20. b. Boucle while équivalente. c. Même chose de 20 à 2, par ordre décroissant.

Méthode, pas à pas

Étape 1 — Pour une boucle bornée (for) avec un pas régulier, range() accepte trois arguments : le début, la fin (exclue), et le pas. Quel pas permet de ne prendre que les nombres pairs ?

Étape 2 — Pour la version while : quelles sont les 4 briques indispensables ? Une variable de départ, un test d'arrêt, le print, et une instruction qui fait progresser la variable — dans quel ordre les placer ?

Étape 3 — Pour aller de 20 à 2 en ordre décroissant avec un for, le pas doit être négatif. Que se passe-t-il si tu oublies ce signe négatif ?

Exercice 18

La fonction random() du module random produit un nombre aléatoire décimal entre 0 et 1. Écris une fonction qui affiche aléatoirement "Haha", "Hahaha", ... jusqu'à "Hahahaha" (entre 1 et 10 répétitions de "ha").

Méthode, pas à pas

Étape 1 — Commence par from random import random pour pouvoir utiliser cette fonction.

Étape 2 — random() renvoie un nombre entre 0 (inclus) et 1 (exclu). Comment transformer ce nombre décimal en un entier compris entre 1 et 10 ? (Indice : multiplie par 10, arrondis vers le bas avec int(...), puis ajoute 1.)

Étape 3 — Une fois que tu as ton nombre n de répétitions, comment construire une chaîne contenant "ha" répété n fois ? (Indice : la multiplication de chaînes, "ha" * n.)

Étape 4 — N'oublie pas la majuscule sur le premier « h » et le point d'exclamation final si tu veux imiter exactement le modèle donné.

Exercice 19

a. Boucle qui affiche les puissances de 2 inférieures à un million. b. Modifie le programme pour compter ces puissances.

Méthode, pas à pas

Étape 1 — Connais-tu à l'avance le nombre de puissances de 2 à afficher avant de commencer ? Si non, quelle structure de boucle (for ou while) est adaptée ici ?

Étape 2 — Pars d'une variable puissance = 1 (qui vaut 2^0). Quelle opération permet de passer d'une puissance de 2 à la suivante ?

Étape 3 — Quelle condition d'arrêt utiliser pour rester strictement en dessous d'un million ?

Étape 4 — Pour la question b, ajoute une variable compteur, initialisée avant la boucle, et incrémentée à chaque tour.

Exercice 20

Écris un programme qui affiche les chiffres d'un nombre à partir du dernier (exemple pour 1234 : affiche 4, puis 3, puis 2, puis 1).

Méthode, pas à pas

Étape 1 — Comment récupérer uniquement le DERNIER chiffre d'un nombre n avec une opération arithmétique ? (Indice : le reste de la division par 10.)

Étape 2 — Une fois ce dernier chiffre affiché, comment « retirer » ce chiffre du nombre pour pouvoir traiter le suivant au tour d'après ? (Indice : la division entière par 10.)

Étape 3 — Quelle boucle utiliser, et quelle condition d'arrêt (que vaut n lorsqu'il ne reste plus de chiffres à traiter) ?

Exercice 21

a. Affiche les 100 premiers nombres de Fibonacci. b. Modifie pour afficher aussi le rapport entre deux nombres consécutifs.

Méthode, pas à pas

Étape 1 — Rappelle-toi la règle de Fibonacci : chaque terme est la somme des deux précédents. De quelles DEUX valeurs as-tu besoin en mémoire à chaque étape, pas d'une seule ?

Étape 2 — Pars de deux variables $a = 0$ et $b = 1$. Comment calculer le terme suivant à partir de a et b ?

Étape 3 — Astuce Python : l'instruction $a, b = b, a + b$ permet de mettre à jour les deux variables en une seule ligne, sans variable temporaire. Essaie de comprendre pourquoi ça marche avant de l'utiliser.

Étape 4 — Pour la question b, à quel moment peux-tu calculer b / a (le rapport) ? Que se passe-t-il la toute première fois, quand a vaut 0 ?

Exercice 22

Suite de Syracuse : si n est pair, le terme suivant est $n/2$; si n est impair, c'est $3n+1$. On répète jusqu'à atteindre 1.

a. Fonction Syracuse(n) qui affiche la suite. b. Modifie pour afficher aussi le nombre d'étapes. c. Modifie pour renvoyer le plus grand nombre atteint.

Méthode, pas à pas

Étape 1 — Sais-tu à l'avance combien d'étapes seront nécessaires pour atteindre 1 ? Quelle boucle utiliser dans ce cas ?

Étape 2 — À chaque tour, comment teste-t-on si n est pair ou impair ? Puis applique la bonne formule selon le cas (if/else).

Étape 3 — Pour la question b, ajoute un compteur d'étapes, comme dans l'exercice 19b.

Étape 4 — Pour la question c, il te faut une variable qui garde en mémoire le plus grand nombre vu jusqu'ici. Après avoir calculé un nouveau terme, comment la mettre à jour si ce terme est plus grand que le maximum actuel ?

Exercice 23

a. Affiche toutes les combinaisons de deux dés donnant un nombre cible entre 2 et 12. b. Étends pour tous les nombres de 2 à 12. c. Adapte pour trois dés. d. Affiche seulement le nombre de combinaisons.

Méthode, pas à pas

Étape 1 — Un dé peut prendre les valeurs 1 à 6. Pour tester TOUTES les paires (dé1, dé2) possibles, quelle structure utiliser ? (Indice : deux boucles for imbriquées, chacune sur range(1, 7).)

Étape 2 — À l'intérieur des boucles, quelle condition teste si la somme des deux dés est bien égale au nombre cible ?

Étape 3 — Pour la question b, comment répéter ce travail pour chaque cible de 2 à 12 ? (Indice : encore une boucle for autour de tout le reste.)

Étape 4 — Pour la question c (trois dés), combien de boucles imbriquées te faut-il maintenant ?

Étape 5 — Pour la question d, remplace l'affichage de chaque combinaison par l'incrément d'un compteur, puis affiche uniquement ce compteur à la fin.

Exercice 24

a. Fonction qui teste si un nombre est premier. b. Adapte avec une boucle while. c. Optimise en ne testant que jusqu'à $\sqrt{n}$. d. Affiche les M premiers nombres premiers.

Méthode, pas à pas

Étape 1 — Rappel : n est premier s'il n'est divisible par aucun entier entre 2 et n-1. Quelle boucle for et quel test de divisibilité utiliser (voir l'opérateur déjà rencontré à l'exercice 20) ?

Étape 2 — Pour la version b (while) : quel est l'avantage d'un while ici ? (Indice : peux-tu t'arrêter dès qu'un diviseur est trouvé, sans attendre la fin normale d'un for ?)

Étape 3 — Pour la version c : pourquoi suffit-il de tester les diviseurs jusqu'à la racine carrée de n plutôt que jusqu'à n-1 ? (Réfléchis : si $n = a \times b$ avec $a \leq b$, que peut-on dire de a par rapport à $\sqrt{n}$?) Le module math fournit sqrt(n).

Étape 4 — Pour la question d : à partir de ta fonction est_premier, comment parcourir les entiers un par un (2, 3, 4, ...) en ne gardant que les premiers, jusqu'à en avoir trouvé M ?

Exercice 25

a. Fonction jeu(a, b) qui simule pierre-feuille-ciseaux. b. Fonction qui simule plusieurs parties et compte les scores. c. Simule 50 tours et affiche les scores.

Méthode, pas à pas

Étape 1 — Choisis une représentation simple pour les trois choix, par exemple les chaînes "pierre", "feuille",

"ciseaux". Fais un tableau à la main avec les 9 combinaisons possibles de (a, b) et qui gagne à chaque fois.

Étape 2 — Dans `jeu(a, b)`, commence par traiter le cas d'égalité (`a == b`). Ensuite, il te reste 6 cas à traiter : peux-tu les regrouper avec des `or` pour réduire le nombre de `if` (par exemple, toutes les façons dont "pierre" gagne) ?

Étape 3 — Pour la question b, comment choisir un coup au hasard parmi les trois ? (Indice : une liste `['pierre', 'feuille', 'ciseaux']` et la fonction `random.choice`, ou un nombre aléatoire entre 0 et 2 utilisé comme indice.)

Étape 4 — Pour compter les scores, quelles variables dois-tu créer avant la boucle de simulation, et comment les mettre à jour selon le résultat de chaque partie ?

Étape 5 — Pour la question c, il ne te reste plus qu'à répéter cette simulation 50 fois avec une boucle `for`, puis à afficher les scores finaux une fois la boucle terminée.